S. J. P. N. TRUST'S
HIRASUGAR INSTITUTE OF TECHNOLOGY, NIDASOSHI
Accredited at 'A+' Grade by NAAC
Programmes Accredited by NBA: CSE & ECE
Department of Computer Science & Engineering

Module 2:

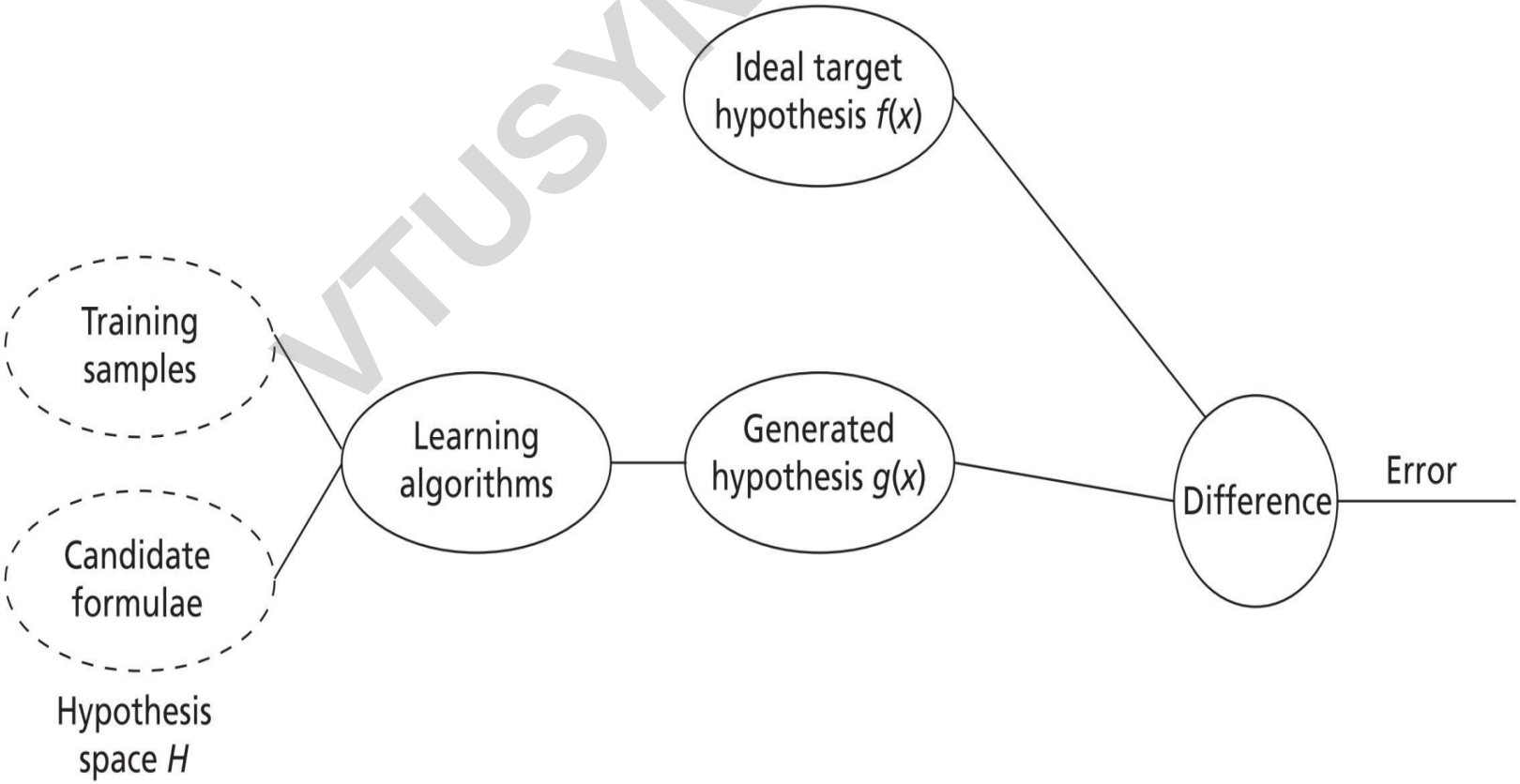
Chapter 3: Basic Learning Theory

Dr. S. V. Manjaragi
Asso. Prof. , Dept. of Computer Science & Engg.,
Hirasugar Institute of Technology, Nidasoshi

Learning

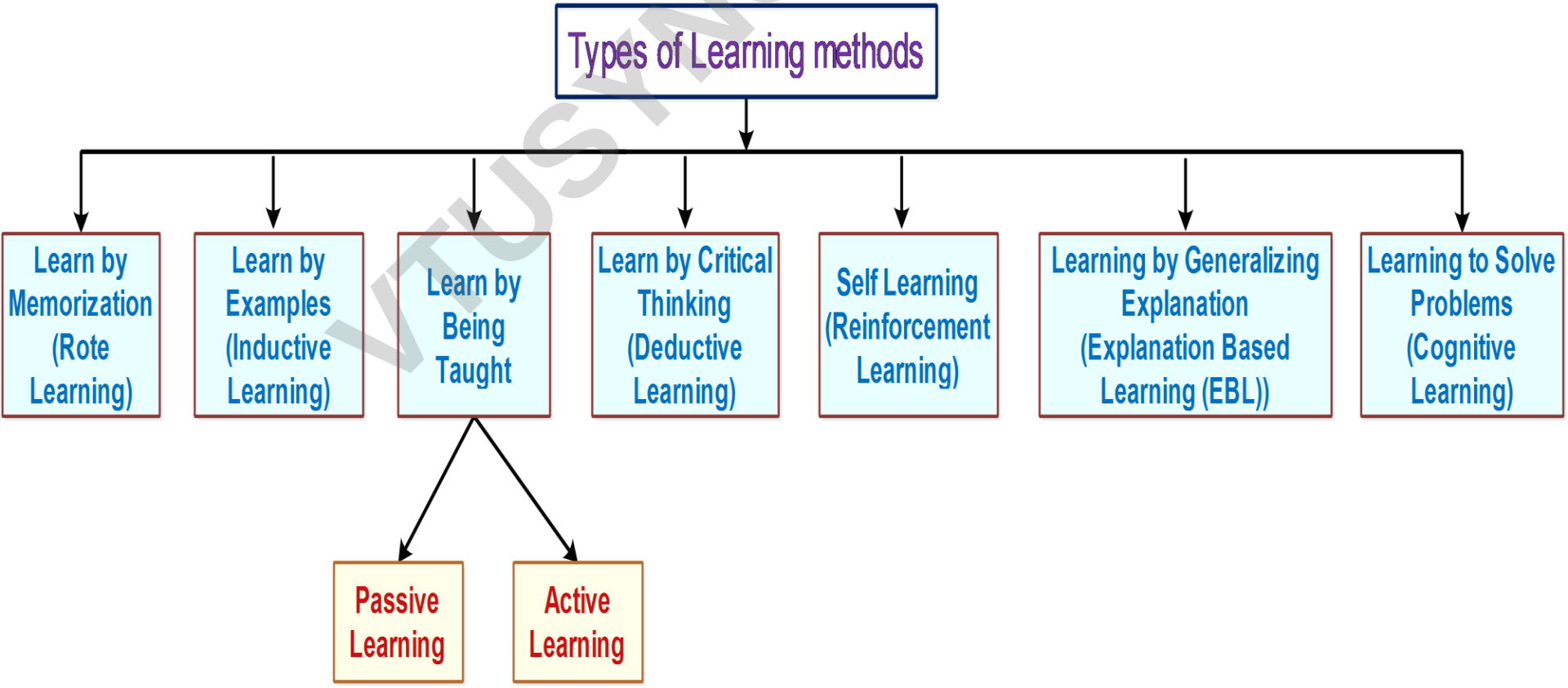
- Learning is a process by which one can **acquire knowledge and construct new ideas or concepts based on the experiences.**
- The standard definition of learning proposed by tom mitchell is that a **program can learn from E for the task T, and P improves with experience** E.
- There are two kinds of problems – well-posed and ill-posed.
- Computers can solve only well-posed problems, as these have **well-defined specifications** and have the following components inherent to it.
- Class of learning tasks (T)
- A measure of performance (P)
- A source of experience (E)

Learning Environment



Learning Model = Hypothesis Set + Learning Algorithm

Learning Types



Introduction to Computation Learning Theory

There are many questions that have been raised by mathematicians and logicians over the time taken by computers to learn. Some of the questions are as follows:

1. How can a learning system predict an unseen instance?
2. How do the hypothesis h is close to f, when hypothesis f itself is unknown?
3. How many samples are required?
4. Can we measure the performance of a learning system?
5. Is the solution obtained local or global?

Design of a Learning System

- **A system that is built around a learning algorithm is called a learning system.**
- The design of systems focuses on these steps:
  - Choosing a training experience
  - Choosing a target function
  - Representation of a target function
  - Function approximation

Introduction to Concept Learning

- Concept learning is a learning strategy of acquiring abstract knowledge or inferring a general concept or deriving a category from the given training samples.
- It is a process of abstraction and generalization from the data.
- Concept learning helps to classify an object that has a set of common, relevant features.
- Concept learning requires three things:
  - **Input -** Training dataset which is a set of training instances, each labeled with the name of a concept or category to which it belongs. Use this past experience to train and build the model.
  - **Output -** Target concept or Target function *f.* It is a mapping function f(x) from input *x to* output y. It is to determine the specific features or common features to identify an object.
  - In other words, it is to *find the hypothesis to determine the target concept.* For e.g., the specific set of features to identify an elephant from all animals.
  - **Test -** New instances to test the learned model.

Introduction to Concept Learning

- Concept learning is defined as **"Given a set of hypotheses, the learner searches through the hypothesis space to identify the best hypothesis that matches the target concept".**

Table 3.1: Sample Training Instances

S.No.	Horns	Tail	Tusks	Paws	Fur	Color	Hooves	Size	Elephant
1.	No	Short	Yes	No	No	Black	No	Big	Yes
2.	Yes	Short	No	No	No	Brown	Yes	Medium	No
3.	No	Short	Yes	No	No	Black	No	Medium	Yes
4.	No	Long	No	Yes	Yes	White	No	Medium	No
5.	No	Short	Yes	Yes	Yes	Black	No	Big	Yes

- Here, in this set of training instances, the independent attributes considered are 'Horns', 'Tail', 'Tusks', 'Paws', 'Fur', 'Color', 'Hooves' and 'Size'. The dependent attribute is 'Elephant'.
- The target concept is to identify the animal to be an Elephant.

Representation of a Hypothesis

- A *hypothesis 'h'* approximates a target function *'f'* to represent the relationship between the independent attributes and the dependent attribute of the training instances.
- The hypothesis is the predicted approximate model that **best maps the inputs to outputs.**
- Each hypothesis is represented as a conjunction of attribute conditions in the antecedent part.

For example, (Tail = Short) $\wedge$ (Color = Black)….

- The set of hypothesis in the search space is called as hypotheses.
- Generally *'H'* is used to represent the hypotheses and *'h'* is used to represent a candidate hypothesis.

Representation of a Hypothesis

- In the antecedent of an attribute condition of a hypothesis, **each attribute can take value as either '?' or *'Ø'* or can hold a single value.**
- "?" denotes that the attribute can take any value [e.g., Color = ?]
- *"Ø"* denotes that the attribute cannot take any value, i.e., it represents a null value [e.g., Horns = *Ø]*
- Single value denotes a specific single value from acceptable values of the attribute, i.e., the attribute 'Tail' can take a value as 'short' [e.g., Tail = Short]

For example, a hypothesis '*h*' will look like,

	Horns	Tail	Tusks	Paws	Fur	Color	Hooves	Size
h =	<No	?	Yes	?	?	Black	No	Medium>

Given a test instance x, we say $h(x) = 1$, if the test instance x satisfies this hypothesis h.

	Horns	Tail	Tusks	Paws	Fur	Color	Hooves	Size
h =	<No	?	Yes	?	?	Black	No	Medium>

(or)

	Horns	Tail	Tusks	Paws	Fur	Color	Hooves	Size
h =	<No	?	Yes	?	?	Black	No	Big>

The task is to predict the best hypothesis for the target concept (an elephant). The most general hypothesis can allow any value for each of the attribute.

It is represented as:

<?, ?, ?, ?, ?, ?, ?, ?>. This hypothesis indicates that any animal can be an elephant.

The most specific hypothesis will not allow any value for each of the attribute <φ, φ, φ, φ, φ, φ, φ, φ>. This hypothesis indicates that no animal can be an elephant.

Example 3.1: Explain Concept Learning Task of an Elephant from the dataset given in Table 3.1.

Given,

Input: 5 instances each with 8 attributes

Target concept/function 'c': Elephant → {Yes, No}

Hypotheses H: Set of hypothesis each with conjunctions of literals as propositions [i.e., each literal is represented as an attribute-value pair]

Solution: The hypothesis 'h' for the concept learning task of an Elephant is given as:

h = <No Short Yes ? ? Black No ?>

This hypothesis h is expressed in propositional logic form as below:

(Horns = No) ∧ (Tail = Short) ∧ (Tusks = Yes) ∧ (Paws = ?) ∧ (Fur = ?) ∧ (Color = Black) ∧ (Hooves = No) ∧ (Size = ?)

Output: Learn the hypothesis 'h' to predict an 'Elephant' such that for a given test instance x,

$$h(x) = c(x)$$

This hypothesis produced is also called as concept description which is a model that can be used to classify subsequent instances.

Hypothesis Space

- Hypothesis space is **the set of all possible hypotheses that approximates the target function f.**
- The subset of hypothesis space that is consistent with all-observed training Instances is called as version space.
- From this set of hypotheses in the hypothesis space, a machine learning algorithm would determine the best possible hypothesis that would best describe the target function or best fit the outputs.
- Every machine learning algorithm would represent the hypothesis space in a different manner about the function that maps the input variables to output variables.
- For example, a **regression algorithm represents the hypothesis space as a linear function whereas a decision tree algorithm represents the hypothesis space as a tree.**

Hypothesis Space

For example, each of the attribute given in the Table 3.1 has the following possible set of values.

Horns - Yes, No

Tail - Long, Short

Tusks - Yes, No

Paws - Yes, No

Fur - Yes, No

Color - Brown, Black, White

Hooves - Yes, No

Size - Medium, Big

Considering these values for each of the attribute, there are $(2 \times 2 \times 2 \times 2 \times 2 \times 3 \times 2 \times 2) = 384$ distinct instances covering all the 5 instances in the training dataset.

So, we can generate $(4 \times 4 \times 4 \times 4 \times 4 \times 5 \times 4 \times 4) = 81{,}920$ distinct hypotheses when including two more values [?, φ] for each of the attribute. However, any hypothesis containing one or more

Heuristic Space Search

- Heuristic search is a search strategy that **finds an optimized hypothesis/solution to a problem by iteratively improving the hypothesis/solution**
- Heuristic search methods will generate a possible hypothesis that can be a solution
- This hypothesis will be tested with the target function or the goal state to see if it is a real solution. If the tested hypothesis is a real solution, them it will be selected.
- commonly used heuristic search methods are **hill climbing methods, constraint satisfaction problems, best-first search, simulated-annealing, A* algorithm, and genetic algorithms.**

Generalization and Specialization

- By generalization of the most specific hypothesis and by specialization of the most general hypothesis, the hypothesis space can be searched for an approximate hypothesis that matches all positive instances but does not match any negative instance.
- ***Searching the Hypothesis Space***

1. Specialization — General to Specific learning
2. Generalization — Specific to General learning

Generalization – Specific to general learning

- This learning methodology will search through the hypothesis space for an approximate hypothesis by **generalizing the most specific hypothesis.**

Table 3.1: Sample Training Instances

S.No.	Horns	Tail	Tusks	Paws	Fur	Color	Hooves	Size	Elephant
1.	No	Short	Yes	No	No	Black	No	Big	Yes
2.	Yes	Short	No	No	No	Brown	Yes	Medium	No
3.	No	Short	Yes	No	No	Black	No	Medium	Yes
4.	No	Long	No	Yes	Yes	White	No	Medium	No
5.	No	Short	Yes	Yes	Yes	Black	No	Big	Yes

Example 3.2: Consider the training instances shown in Table 3.1 and illustrate Specific to General Learning.

The most specific hypothesis is taken now, which will not classify any instance to true.

h = <φ φ φ φ φ φ φ φ>

Read the first instance $I1$, to generalize the hypothesis h so that this positive instance can be classified by the hypothesis $h1$.

$I1$:	No	Short	Yes	No	No	Black	No	Big	**Yes (Positive instance)**
$h1$ =	<No	Short	Yes	No	No	Black	No	Big>	

When reading the second instance $I2$, it is a negative instance, so ignore it.

$I2$:	Yes	Short	No	No	No	Brown	Yes	Medium	**No (Negative instance)**
$h2$ =	<No	Short	Yes	No	No	Black	No	Big>	

Similarly, when reading the third instance $I3$, it is a positive instance so generalize $h2$ to $h3$ to accommodate it. The resulting $h3$ is generalized.

$I3$:	No	Short	Yes	No	No	Black	No	Medium	**Yes (Positive instance)**
$h3$ =	<No	Short	Yes	No	No	Black	No	?>	

Ignore $I4$ since it is a negative instance.

$I4$:	No	Long	No	Yes	Yes	White	No	Medium	**No (Negative instance)**
$h4$ =	<No	Short	Yes	No	No	Black	No	?>	

When reading the fifth instance $I5$, $h4$ is further generalized to $h5$.

$I5$:	No	Short	Yes	Yes	Yes	Black	No	Big	**Yes (Positive instance)**
$h5$ =	<No	Short	Yes	?	?	Black	No	?>	

Now, after observing all the positive instances, an approximate hypothesis $h5$ is generated which can now classify any subsequent positive instance to true.

Specialization – general to specific learning

- This learning methodology will search through the hypothesis space for an approximate hypothesis by specializing the most general hypothesis.

Dataset Example

- Let’s say we’re trying to learn the concept of a **"Fruit that is safe to eat"** based on some features.
- **Attributes:**
- Color: Red, Green, Yellow
- Size: Small, Medium, Large
- Shape: Round, Oval

Training Examples:

Example	Color	Size	Shape	Safe to Eat?
1	Red	Small	Round	Yes
2	Green	Small	Round	Yes
3	Red	Large	Oval	No

- Start with: H0 = (?, ?, ?) This accepts **everything**.
- **After Example 1 (Red, Small, Round, Yes):** We specialize H to fit this positive example:
- H1 = (Red, Small, Round)
- **After Example 2 (Green, Small, Round, Yes):** To generalize H1 to also include this example:
- H2 = (?, Small, Round)
- **After Example 3 (Red, Large, Oval, No):**
- We specialize H2 to exclude this negative example.
- H2 = (?, Small, Round) → includes Example 3 because it doesn't constrain Size or Shape enough.
- But Example 3 has **Large** size and **Oval** shape.
- So we need to **specialize H2** to exclude this combination. Possible specializations of H2:

(Color, Size, Shape):

1. (?, Small, Round) ← current (too general)

→ Try specializing to:

2. (?, Small, ?) → still includes negative

3. (?, ?, Round) → still includes negative

→ Best: (?, Small, Round) already excludes (Large, Oval)

Hypothesis Space Search by Find-S Algorithm

- Find-S algorithm is guaranteed to **converge to the most specific hypothesis in *H* that is consistent with the positive instances** in the training dataset.
- Thus, this algorithm considers only the positive instances and eliminates negative instances while generating the hypothesis.
- It initially starts with the most specific hypothesis.

Algorithm 3.1: Find-S

Input: Positive instances in the Training dataset

Output: Hypothesis *'h'*

1. Initialize *'h'* to the most specific hypothesis.

   $h = <\varphi \quad \varphi \quad \varphi \quad \varphi \quad \varphi \quad \ldots..>$

2. Generalize the initial hypothesis for the first positive instance [Since *'h'* is more specific].
3. For each subsequent instances:

   If it is a positive instance,

   Check for each attribute value in the instance with the hypothesis *'h'*.

   If the attribute value is the same as the hypothesis value, then do nothing,

   Else if the attribute value is different than the hypothesis value, change it to '?' in *'h'*.

   Else if it is a negative instance,

   Ignore it.

- Consider the training dataset of 4 instances shown in Table 3.2.
- It contains the details of the performance of students and their likelihood of getting a job offer or not in their final semester.
- Apply the Find-S algorithm

Table 3.2: Training Dataset

CGPA	Interactiveness	Practical Knowledge	Communication Skills	Logical Thinking	Interest	Job Offer
≥9	Yes	Excellent	Good	Fast	Yes	Yes
≥9	Yes	Good	Good	Fast	Yes	Yes
≥8	No	Good	Good	Fast	No	No
≥9	Yes	Good	Good	Slow	No	Yes

Solution:

Step 1: Initialize '*h*' to the most specific hypothesis. There are 6 attributes, so for each attribute, we initially fill 'φ' in the initial hypothesis '*h*'.

$h = <\varphi \quad \varphi \quad \varphi \quad \varphi \quad \varphi \quad \varphi>$

Step 2: Generalize the initial hypothesis for the first positive instance. *I*1 is a positive instance, so generalize the most specific hypothesis '*h*' to include this positive instance. Hence,

*I*1: ≥9 Yes Excellent Good Fast Yes **Positive instance**

h = <≥9 Yes Excellent Good Fast Yes>

Step 3: Scan the next instance *I*2, since *I*2 is a positive instance. Generalize '*h*' to include positive instance *I*2. For each of the non-matching attribute value in '*h*' put a '?' to include this positive instance. The third attribute value is mismatching in '*h*' with *I*2, so put a '?'.

*I*2: ≥9 Yes Good Good Fast Yes **Positive instance**

h = <≥9 Yes ? Good Fast Yes>

Now,scan *I*3. Since it is a negative instance, ignore it. Hence, the hypothesis remains the same without any change after scanning *I*3.

I3: ≥8 No Good Good Fast No **Negative instance**

h = <≥9 Yes ? Good Fast Yes>

Now scan *I*4. Since it is a positive instance, check for mismatch in the hypothesis '*h*' with *I*4. The 5th and 6th attribute value are mismatching, so add '?' to those attributes in '*h*'.

I4: ≥9 Yes Good Good Slow No **Positive instance**

h = <≥9 Yes ? Good ? ?>

Now, the final hypothesis generated with Find-S algorithm is:

h = <≥9 Yes ? Good ? ?>

It includes all positive instances and obviously ignores any negative instance.

Limitations of Find-S Algorithm

- Find-S algorithm tries to find a hypothesis that is consistent with positive instances, ignoring all negative instances. **As long as the training dataset is consistent, the hypothesis found by this algorithm may be consistent.**
- The algorithm finds only one unique hypothesis, wherein there may be many other hypotheses that are consistent with the training dataset.
- Many times, the training dataset may contain some errors; hence such inconsistent data instances can mislead this algorithm in determining the consistent hypothesis since it ignores negative instances.

Version Spaces

- The version space contains the subset of hypotheses from the hypothesis space that is consistent with all training instances in the training dataset.

List-Then-Eliminate Algorithm

- The principle idea of this learning algorithm is to initialize the version space **to contain all hypotheses and then eliminate any hypothesis that is found inconsistent with any training instances.**

Algorithm 3.2: List-Then-Eliminate

Input: Version Space – a list of all hypotheses

Output: Set of consistent hypotheses

1. Initialize the version space with a list of hypotheses.
2. For each training instance,
   - remove from version space any hypothesis that is inconsistent.

Version Spaces and the Candidate Elimination Algorithm

- Version space learning is to generate all consistent hypotheses around.
- This algorithm computes the version space by the combination of the two cases namely,
  - Specific to General learning — Generalize S to include the positive example
  - General to Specific learning — Specialize *G* to exclude the negative example
- The algorithm defines two boundaries called ***'general boundary'*** which is a set of all hypotheses that are the most general
- and ***'specific boundary'*** which is a set of all hypotheses that are the most specific.
- Thus, the algorithm limits the version space to contain only those hypotheses that are most general and most specific.

Candidate Elimination Algorithm

Algorithm 3.3: Candidate Elimination

Input: Set of instances in the Training dataset

Output: Hypothesis *G* and *S*

1. Initialize *G*, to the maximally general hypotheses.
2. Initialize *S*, to the maximally specific hypotheses.
   - Generalize the initial hypothesis for the first positive instance.
3. For each subsequent new training instance,
   - If the instance is **positive**,
     - Generalize *S* to include the positive instance,
       - Check the attribute value of the positive instance and *S*,
         - If the attribute value of positive instance and *S* are different, fill that field value with '?'.
         - If the attribute value of positive instance and *S* are same, then do no change.
     - Prune *G* to exclude all inconsistent hypotheses in *G* with the positive instance.
   - If the instance is **negative,**
     - Specialize *G* to exclude the negative instance,
       - Add to *G* all minimal specializations to exclude the negative example and be consistent with *S*.
         - If the attribute value of *S* and the negative instance are different, then fill that attribute value with *S* value.
         - If the attribute value of *S* and negative instance are same, no need to update '*G*' and fill that attribute value with '?'.
     - Remove from *S* all inconsistent hypotheses with the negative instance.

Table 3.2: Training Dataset

CGPA	Interactiveness	Practical Knowledge	Communication Skills	Logical Thinking	Interest	Job Offer
≥9	Yes	Excellent	Good	Fast	Yes	Yes
≥9	Yes	Good	Good	Fast	Yes	Yes
≥8	No	Good	Good	Fast	No	No
≥9	Yes	Good	Good	Slow	No	Yes

Example 3.4: Consider the same set of instances from the training dataset shown in Table 3.3 and generate version space as consistent hypothesis.

Solution:

Step 1: Initialize '*G*' boundary to the maximally general hypotheses,

$G = <?$? ? ? ? ?>

Step 2: Initialize '*S*' boundary to the maximally specific hypothesis. There are 6 attributes, so for each attribute, we initially fill 'φ' in the hypothesis '*S*'.

$S = <\varphi$ φ φ φ φ $\varphi>$

Generalize the initial hypothesis for the first positive instance. *I*1 is a positive instance; so generalize the most specific hypothesis '*S*' to include this positive instance. Hence,

*I*1: ≥9 Yes Excellent Good Fast Yes **Positive instance**

*S*1 = < ≥9 Yes Excellent Good Fast Yes>

*G*1 = <? ? ? ? ? ?>

Step 3:

Iteration 1

Scan the next instance *I*2. Since *I*2 is a positive instance, generalize '*S*1' to include positive instance *I*2. For each of the non-matching attribute value in '*S*1', put a '?' to include this positive instance. The third attribute value is mismatching in '*S*1' with *I*2, so put a '?'.

*I*2:	≥9	Yes	Good	Good	Fast	Yes	**Positive instance**
*S*2 = <	≥9	Yes	?	Good	Fast	Yes>	

Prune *G*1 to exclude all inconsistent hypotheses with the positive instance. Since *G*1 is consistent with this positive instance, there is no change. The resulting *G*2 is,

*G*2 = <?	?	?	?	?	?>	

Iteration 2

Now Scan *I*3,

*I*3:	≥8	No	Good	Good	Fast	No	**Negative instance**

Since it is a negative instance, specialize *G*2 to exclude the negative example but stay consistent with *S*2. Generate hypothesis for each of the non-matching attribute value in *S*2 and fill with the attribute value of *S*2. In those generated hypotheses, for all matching attribute values, put a '?'. The first, second and 6th attribute values do not match, hence '3' hypotheses are generated in *G*3.

There is no inconsistent hypothesis in *S*2 with the negative instance, hence *S*3 remains the same.

*G*3 = <	≥9	?	?	?	?	?>
	<?	Yes	?	?	?	?>
	<?	?	?	?	?	Yes>
*S*3 = <	≥9	Yes	?	Good	Fast	Yes>

Iteration 3

Now Scan *I*4. Since it is a positive instance, check for mismatch in the hypothesis '*S*3' with *I*4. The 5th and 6th attribute value are mismatching, so add '?' to those attributes in '*S*4'.

*I*4: ≥9	Yes	Good	Good	Slow	No	**Positive instance**
*S*4 = <≥9	Yes	?	Good	?	?>	

Prune *G*3 to exclude all inconsistent hypotheses with the positive instance *I*4.

*G*3 = <≥9	?	?	?	?	?>	
<?	Yes	?	?	?	?>	
<?	?	?	?	?	Yes>	**Inconsistent**

Since the third hypothesis in *G*3 is inconsistent with this positive instance, remove the third one. The resulting *G*4 is,

*G*4 = <≥9	?	?	?	?	?>
<?	Yes	?	?	?	?>

Using the two boundary sets, S4 and G4, the version space is converged to contain the set of consistent hypotheses.

The final version space is,

<≥9	Yes	?	?	?	?>
<≥9	?	?	Good	?	?>
<?	Yes	?	Good	?	?>

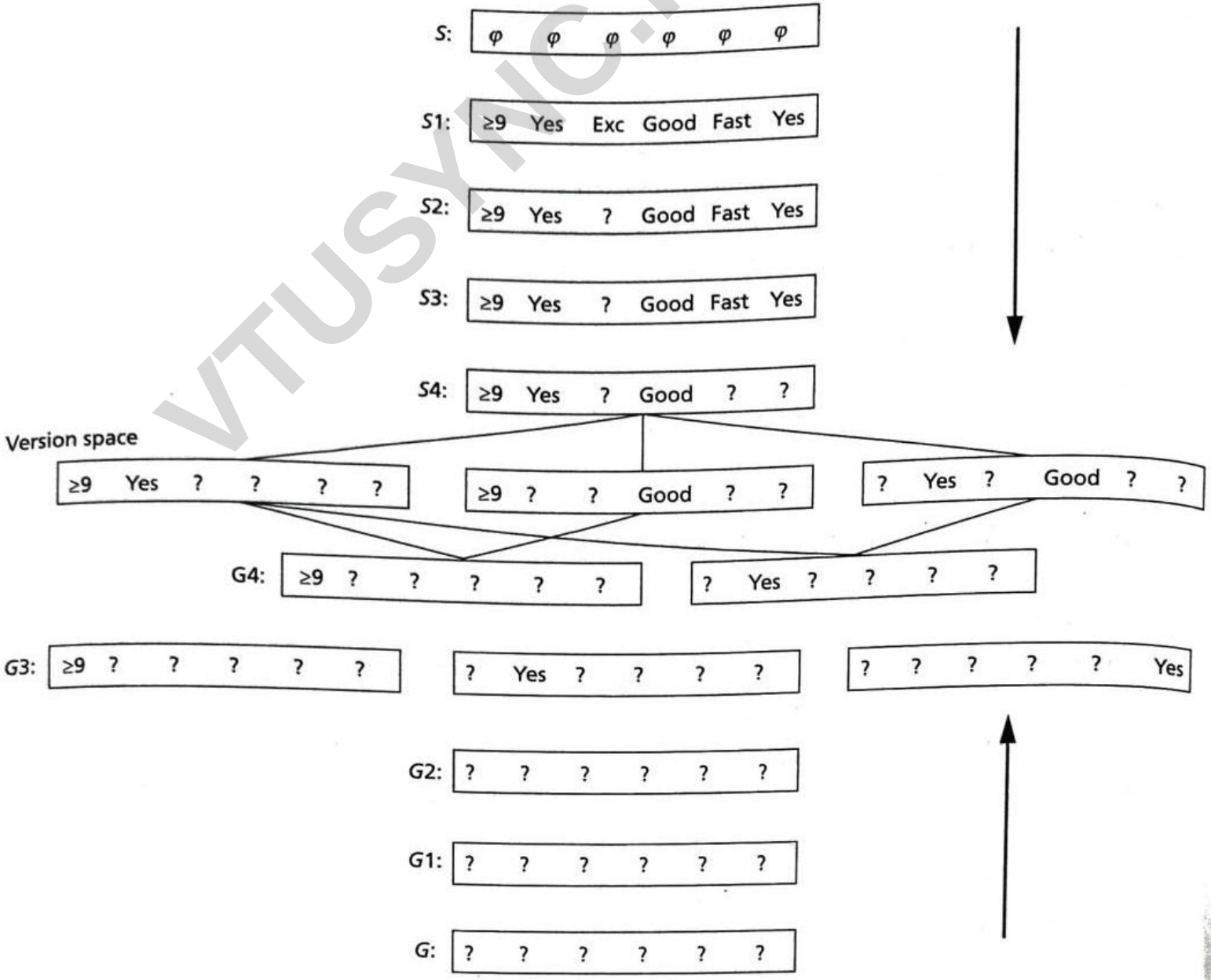


Figure 3.2: Deriving the Version Space

Modelling in Machine Learning

- A machine learning model is an abstraction of the training dataset that can perform a prediction on new data.
- Training the model means feeding instances to the machine learning algorithm.
- Training datasets are used to fit and tune the model.
- After training a machine learning algorithm with the training data, a predictive model is generated as output to which a new data is fed to make predictions.
- The **process of modelling means training a machine learning algorithm with the training dataset, tuning it to increase performance, validating it and making predictions for a new unseen data.**
- The major concern in machine learning **is what model to select, how to train the model, time required to train, the dataset to be used, what performance to expect,** and so on.

Machine Learning Process

- The four basic steps in the machine learning process are:

1. Choose a machine learning algorithm to suit the training data and the problem domain
2. Input the training dataset and train the machine learning algorithm to learn from the data and capture the patterns in the data
3. Tune the parameters of the model to improve the accuracy of learning of the algorithm
4. Evaluate the learned model once the model is built

Re-sampling Methods

- **Re-sampling is a technique to select a model by reconstructing the training dataset and test dataset by randomly choosing instances by some method from the given dataset.**
- This method involves selecting different instances repeatedly from a training dataset to tune a model.
- It is done to improve the accuracy of a model.
- The common re-sampling model selection methods are **Random tram/test splits, Cross-Validation (K-fold, LOOCV, etc.) and Bootstrap.**

Cross-Validation

- Cross-Validation is a **method by which we can tune the model with only training dataset.**
- It is a model **evaluation approach** by which **we can set aside some data of the training dataset for validation and fit the rest of the data to train the model.**
- The best model is found by estimating the average of errors on different test data.
- The popular cross-validation family of methods includes **Holdout method, K-fold cross-validation, Stratified cross-validation and Leave-One-Out Cross-Validation (LOOCV).**

Holdout Method

- This is the simplest method of cross-validation.
- The dataset is split into two subsets called training dataset and test dataset. The model is trained using the training dataset and then evaluated using the test dataset.
- This holdout method can be applied for a single time which is called as **single holdout method**
- or it can be repeated for more than once which is called as **repeated holdout method**.
- The average performance on the test dataset is estimated to evaluate the model.
- Even though this model is very simple, **it can exhibit high variance and the performance largely depends on how the dataset is split.**

K-fold Cross-Validation

- It will **split the training dataset into *k* equal folds/parts creating *k* — 1 subsets of training set and one test subset.**
- Out of the *k* folds, *k* — 1 folds are used for training and one fold is used for testing the model.
- This has to be performed for ***k* iterations and during each iteration a different fold is selected for testing.**
- The average performance of the model on *k* iterations is the final estimate of the model performance.

K-fold Cross-Validation

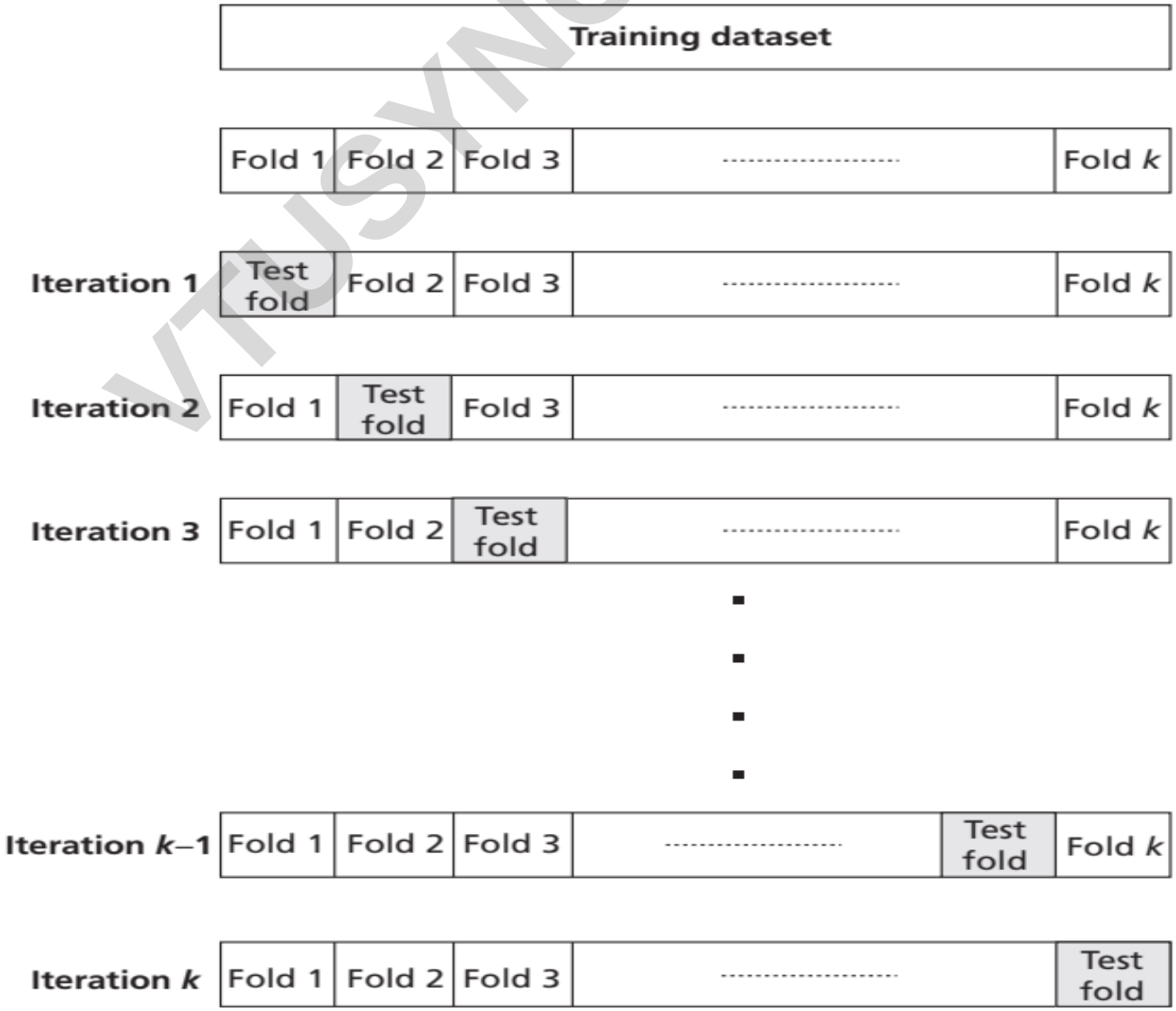


Figure 3.4: Illustration of *K*-fold Cross-Validation

K-fold Cross-Validation

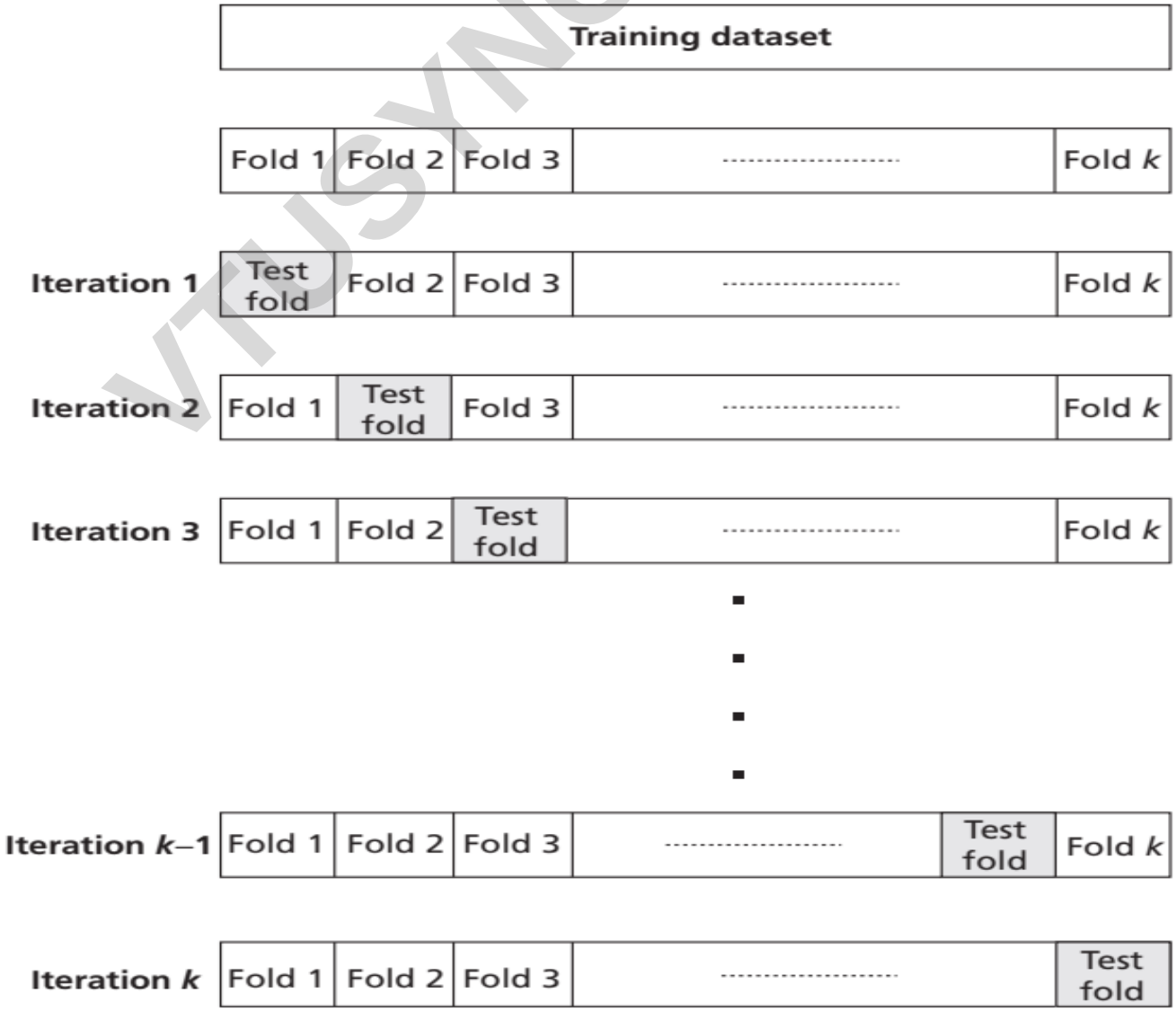


Figure 3.4: Illustration of *K*-fold Cross-Validation

K-fold Cross-Validation

Algorithm 3.4: *K*-fold Cross Validation

Input: Dataset with '*n*' instances

'*k*' – Number of folds

1. Repeat the following steps for '*k*' times with a different 'hold-out' fold for evaluation:
   - Split the dataset into *k* equal folds.
   - Train the model on $(k-1)$ folds.
   - Evaluate it on the 1 remaining "hold-out" fold.
2. Compute average of the performance across all *k* hold-out folds.

Stratified K-fold Cross-Validation

- This method is similar to k-fold cross-validation but with a slight difference.
- Here, it is ensured that **while splitting the dataset into *k* folds, each fold should contain the same proportion of instances with a given categorical value. This is called stratified cross-validation.**

Leave-One-Out Cross-Validation (LOOCV)

- This method repeatedly **splits the n data instances of the dataset into training dataset containing n — 1 data instances and leaving one data instance for evaluating the model.**
- This process is repeated n times and average test error is then estimated for the model.
- Even though this **model is expensive and time consuming** because it has to run for n times (i.e., *n* data instances in the dataset), **it has less bias.**

Leave-One-Out Cross-Validation (LOOCV)

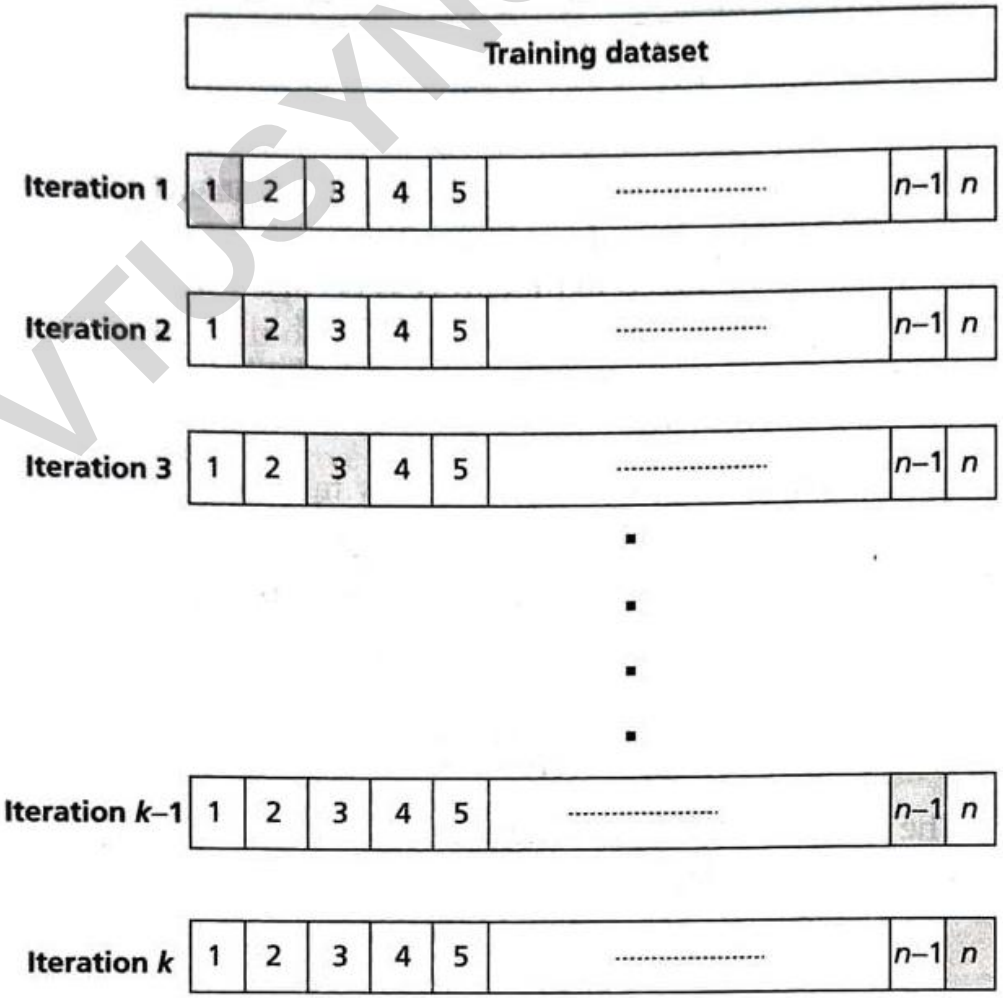


Figure 3.5: Illustration of Leave-One-Out Cross-Validation

Leave-One-Out Cross-Validation (LOOCV)

Algorithm 3.5: LOOCV

Input Dataset with '*n*' instances:

1. Repeat the following steps for '*n*' times with a different 'hold-out' data instance for evaluation:
   - Train the model on $(n-1)$ data instances.
   - Evaluate it on the one remaining 'hold-out' test instance.
2. Compute average of the performance across all *n* holdouts.

Model Performance

- One way to compute the metrics is to form a table called **contingency table.**

Table 3.4: Contingency Table

Test vs Disease	Has Disease Cancer	Has No Disease As Cancer
Positive	True Positive	False Positive
Negative	False Negative	True Negative

- **True Positive (TP) =** Number of cancer patients who are classified by the test correctly,
- **True Negative (TN) =** Number of normal patients who do not have cancer are correctly detected.
- **The two errors** that are involved in this process:
- **False Positive (FP)** that is an alarm that indicates that the tests show positive when the patient has no disease
- and **False Negative (FN) :** another error that says a patient has cancer when tests says negative or normal.
- FP and FN are costly errors in this classification process.

Model Performance

1. Sensitivity – The sensitivity of a test is the probability that it will produce a true positive result when used on a test dataset. It is also known as true positive rate. The sensitivity of a test can be determined by calculating:

$$\frac{TP}{TP + FN}$$

2. Specificity – The specificity of a test is the probability that a test will produce a true negative result when used on test dataset.

$$\frac{TN}{TN + FP}$$

3. Positive Predictive Value – The positive predictive value of a test is the probability that an object is classified correctly when a positive test result is observed.

$$\frac{TP}{TP + FP}$$

4. Negative Predictive Value – The negative predictive value of a test is the probability that an object is not classified properly when a negative test result is observed.

$$\frac{TN}{TN + FN}$$

Model Performance

5. Accuracy – The accuracy of the classifier can be shown in terms of sensitivity computed as:

$$\frac{TP + TN}{TP + TN + FP + FN}$$

6. Precision – Precision is also known as positive predictive power. It is defined as the ratio of true positive divided by the sum of true positive and false positive.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Precision indicates how good classifier is in predicting the positive classes.

7. Recall – It is same as sensitivity.

$$\text{Recall} = \text{Sensitivity} = \frac{TP}{TP + FN}$$

A combination of harmonic mean of precision and recall is called *F*-measure or *F*1 score. This is useful in identifying the model skill for a specific threshold.

Visual Classifier Performance

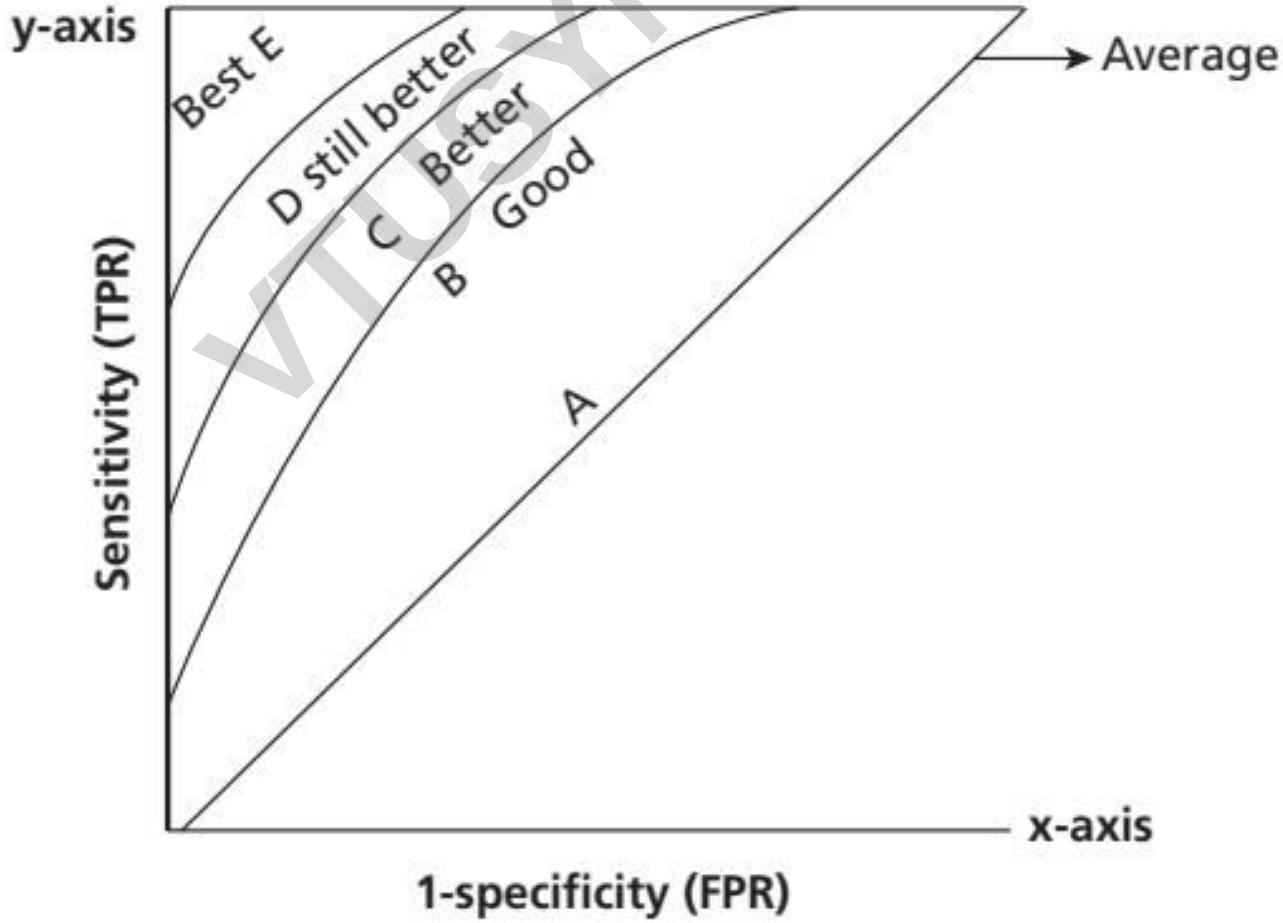


Figure 3.6: A Sample ROC Curve